Fundamentos de la POO

Objetivos

- Comprender algo

Contenido

Introducción 3

Introducción

Hasta este punto de nuestro aprendizaje en el módulo de Programación, hemos construido algoritmos apoyándonos en los pilares de la programación estructurada e imperativa. A lo largo de los temas anteriores, aprendimos a declarar variables de tipos primitivos, a evaluar expresiones numéricas y relacionales, a modificar el **flujo de control** mediante estructuras de selección y repetición, y a interactuar de forma básica con el usuario o la biblioteca estándar del lenguaje. Sin embargo, si analizamos los programas elaborados en esa etapa, observaremos que todos ellos abordan los problemas manipulando datos desagregados y subprogramas aislados que operan de forma lineal sobre dichos datos.

Cuando la dimensión y la complejidad de los problemas del mundo real aumentan, el modelo estructurado tradicional empieza a mostrar una nueva dificultad: **cómo organizar el programa para que su estructura siga siendo comprensible y fácil de mantener**. Gestionar muchas variables sueltas de tipo cadena, entero o real para el nombre, la edad o la nota de múltiples alumnos, por ejemplo, realizando muchas operaciones diferentes sobre cada una, manteniendo las variables y las instrucciones de forma independiente, da lugar a un código complejo y propenso a errores. Además, debemos garantizar que las operaciones se realizan con el alumno correcto y que sus datos conservan en todo momento la coherencia necesaria. La razón fundamental de esta limitación radica en que la programación clásica no relaciona la información con las operaciones que se realizan sobre ella.

Para superar esta barrera metodológica surge el paradigma de **programación orientada a objetos** (habitualmente, POO). Como ya vimos al estudiar los conceptos básicos del lenguaje Java, un objeto puede representar una entidad mediante un conjunto de datos y las operaciones asociadas a los mismos. La POO propone un cambio de paradigma: en lugar de traducir el dominio de un problema a una secuencia de instrucciones sobre tipos primitivos, el software se organiza modelando entidades del mundo real mediante **objetos**. Ahora profundizaremos en esta idea para comprender cómo se construyen los objetos, cómo se relacionan entre sí y cómo podemos utilizarlos para organizar programas de mayor complejidad. También veremos qué dificultades presenta la programación estructurada cuando los problemas crecen y qué mecanismos proporciona la orientación a objetos para abordar esas dificultades.

Java es un lenguaje orientado a objetos y, aunque ya hemos utilizado clases y objetos proporcionados por su biblioteca estándar, hasta ahora nos hemos centrado principalmente en las instrucciones que permiten resolver algoritmos. A lo largo de este tema 4, aprenderemos a definir nuestras propias clases, crear objetos a partir de ellas y establecer qué información almacenan y qué operaciones ofrecen. Para ello, estudiaremos los conceptos de estado, comportamiento e identidad, así como la abstracción y la encapsulación, que permiten construir representaciones de los elementos del problema y controlar la forma en que se accede a ellos. También conoceremos cómo se crean e inicializan los objetos, cómo se gestionan sus referencias y qué papel desempeñan los constructores y los métodos.

El objetivo no es abandonar las técnicas de programación que hemos aprendido, sino incorporarlas a una forma de organización más adecuada para determinados problemas. Las secuencias, las selecciones y las repeticiones seguirán siendo necesarias para expresar los algoritmos; la orientación a objetos nos permitirá, además, organizar los datos y las operaciones que forman parte de esos algoritmos. Con ello comenzaremos a construir programas en los que cada clase tenga unas responsabilidades definidas y cuyos objetos puedan colaborar para resolver un problema común.

El paradigma orientado a objetos

Hasta ahora hemos aprendido a resolver problemas mediante algoritmos y a traducirlos a instrucciones que el ordenador puede ejecutar. Hemos utilizado variables para representar datos, expresiones para operar con ellos y estructuras de control para determinar qué instrucciones deben ejecutarse y en qué orden. Sin embargo, existe otra cuestión que debemos abordar cuando desarrollamos aplicaciones de mayor complejidad: cómo organizar los distintos elementos que componen un programa para representar adecuadamente el problema que estamos resolviendo.

Un **paradigma de programación** representa un marco de pensamiento, una perspectiva que determina cómo el desarrollador modela la realidad, estructura el programa y organiza el código. No se trata únicamente de una colección de instrucciones o de una sintaxis particular, sino de un conjunto de ideas y principios que orientan la forma en que representamos los problemas y construimos sus soluciones. A lo largo de la historia de la informática han surgido diferentes paradigmas, cada uno de ellos con sus propias formas de abordar la programación. Estos paradigmas no han surgido por capricho, sino como respuesta al tipo de problemas a resolver y a la complejidad implícita en los mismos.

En el paradigma de la programación imperativa, un programa se concibe como una jerarquía descendente de algoritmos y subprogramas que operan sobre estructuras de datos mantenidas de forma independiente. En la programación estructurada tradicional, los procedimientos dependen de forma crítica de la representación interna de los datos; por consiguiente, cualquier modificación en la estructura de una variable se propaga en cadena a lo largo de decenas de subprogramas, comprometiendo la estabilidad del código y dificultando la reutilización.

El **paradigma de la Programación Orientada a Objetos (POO)** propone una transformación conceptual. Su propuesta consiste en organizar los programas en torno a objetos que representan entidades relevantes para el problema, reuniendo la información que las describe y las operaciones que pueden realizarse sobre ella. En lugar de pensar exclusivamente en una sucesión de instrucciones que manipulan datos, podemos concebir el sistema como un conjunto de entidades que mantienen su propia información, los **objetos**, y colaboran para alcanzar un objetivo común.

Para comprender por qué surgió esta forma de programación y cuáles son las ideas que la sustentan, comenzaremos por su evolución histórica. A continuación, estudiaremos las dificultades que aparecen cuando los programas crecen y los fundamentos conceptuales que permiten comprender cómo se organizan los objetos.

Evolución histórica y filosofía de la POO

Los primeros lenguajes de programación se diseñaron para expresar cálculos y procedimientos que el ordenador debía ejecutar. El programador describía los datos necesarios y las instrucciones que debían manipularlos, organizando la solución mediante secuencias de operaciones y, posteriormente, mediante procedimientos y funciones. Este enfoque permite resolver muchos problemas de forma eficaz, pero no siempre resulta natural cuando debemos representar sistemas formados por numerosas entidades que mantienen información propia e interactúan unas con otras.

La consolidación de la Programación Orientada a Objetos no fue una invención repentina, sino el fruto de más de tres décadas de investigación en el diseño de lenguajes de programación, la arquitectura de sistemas y las interfaces hombre-máquina.

Simula

Desarrollado en el Centro Noruego de Computación en Oslo por Ole-Johan Dahl y Kristen Nygaard como una extensión del lenguaje ALGOL 60, fue diseñado para resolver problemas de simulación de eventos discretos (modelado de sistemas físicos como el tráfico marítimo en un puerto, colas de espera o redes de transporte), es decir, sistemas cuyo comportamiento evoluciona a través de una sucesión de acontecimientos.

La primera versión, Simula I, apareció a mediados de los años sesenta. Posteriormente, en 1967, se presentó Simula 67, una evolución del lenguaje que incorporó conceptos fundamentales para la programación orientada a objetos. Entre sus aportaciones destacaron las **clases**, los **objetos**, las relaciones de **herencia** y los **procedimientos virtuales**. Estos mecanismos permitían representar entidades con características comunes y comportamientos específicos, así como organizar las relaciones entre ellas.

La importancia de Simula no reside tanto en haber proporcionado nuevas instrucciones al programador, sino en mostrar que muchos problemas pueden expresarse de forma más natural si el programa representa las entidades que intervienen en el sistema. Cada entidad puede mantener un estado propio y disponer de operaciones asociadas. Por esta razón, Simula 67 se considera habitualmente el primer lenguaje de programación orientado a objetos. Aunque su propósito inicial estaba relacionado con la simulación, las ideas que introdujo demostraron tener una utilidad mucho más amplia y terminaron influyendo en numerosos lenguajes posteriores.[1]

Smalltalk

Durante la década de 1970, los investigadores del Palo Alto Research Center (Xerox PARC) crearon Smalltalk, considerado el primer lenguaje y entorno «puro» orientado a objetos. Entre ellos destacó Alan Kay, quien acuñó el término *Object-Oriented*

1. J. Sklenar, *Introduction to Simula*, Universidad de Malta. Disponible en: https://staff.um.edu.mt/jskl1/talk.html. Véase también el capítulo sobre Simula 67 en *General Concepts of the Simula 67 Programming Language*, de J. D. Ichbiah y S. P. Morse.

Programming, junto con otros investigadores como Dan Ingalls y Adele Goldberg, que participaron en la evolución de Smalltalk.

Smalltalk contribuyó a consolidar una visión de la programación en la que el sistema se organiza mediante objetos que interactúan enviándose mensajes. Un objeto no se limita a almacenar información: también ofrece un conjunto de operaciones a través de las cuales otros objetos pueden solicitarle que realice determinadas tareas.

Esta perspectiva introduce una idea especialmente importante: **la solución de un problema puede construirse mediante la colaboración entre entidades con responsabilidades diferenciadas**. Cada objeto participa en el funcionamiento general del programa sin que sea necesario que los demás conozcan todos los detalles de su funcionamiento interno.

Smalltalk fue el precursor de las interfaces gráficas de usuario (GUI). Su evolución durante los años setenta y ochenta contribuyó a extender las ideas de la POO y a mostrar que podían aplicarse a la construcción de sistemas informáticos de propósito general.[2]

La extensión de la orientación a objetos

A principios de la década de los ochenta, Bjarne Stroustrup desarrolló **C++ («C con clases»)** en los Laboratorios Bell, añadiendo mecanismos de abstracción de datos y jerarquías de objetos sobre la base del lenguaje C. Su desarrollo impulsó la adopción industrial del paradigma, permitiendo escribir aplicaciones de alto rendimiento mientras se disfrutaba de los beneficios de la programación orientada a objetos.

Paradigma de programación: Enfoque conceptual, principios y modelo, que determina cómo se estructura el código, se organiza la lógica y se representa el dominio de un problema.

Simula 67: Primer lenguaje de programación que introdujo las nociones formales de la POO para la simulación de sistemas físicos.

Smalltalk: Primer lenguaje y entorno completamente orientado a objetos, precursor de las interfaces gráficas de usuario y del principio de comunicación por paso de mensajes.

Paso de mensajes: Mecanismo conceptual mediante el cual un objeto solicita a otro que ejecute una operación (invoque un método), pasando los parámetros requeridos.

Durante las décadas siguientes, se incorporaron a numerosos lenguajes los principios de la programación orientada a objetos . C++, como acabamos de ver, amplió un lenguaje ya existente para incorporar mecanismos orientados a objetos, línea seguida por muchos otros; otros sin embargo, como Eiffel y Java, desarrollaron sus propias propuestas para organizar el software mediante clases y objetos.

---

2. Alan C. Kay, *The Early History of Smalltalk*, ACM SIGPLAN Notices, vol. 28, n.º 3, 1993. DOI: https://doi.org/10.1145/155360.155364. Véase también *Kids and Us: The Story of Smalltalk*, IEEE Spectrum: https://spectrum.ieee.org/xerox-parc/smalltalk-history.

La expansión de este paradigma estuvo relacionada con la necesidad de construir aplicaciones cada vez más grandes, en las que resultaba importante reutilizar componentes, distribuir responsabilidades y controlar las dependencias entre las distintas partes del sistema. La orientación a objetos proporcionaba mecanismos que ayudaban a describir estas relaciones y a organizar el código de manera que pudiera evolucionar con el tiempo.

Java, presentado en 1995 por James Gosling y Sun Microsystems, adoptó las clases y los objetos como elementos fundamentales para organizar los programas, eliminando elementos complejos y propensos a errores (como el uso de punteros, la gestión manual de memoria o la herencia múltiple). Hasta ahora hemos utilizado objetos y clases proporcionados por su biblioteca estándar, pero nos hemos centrado principalmente en las instrucciones necesarias para resolver algoritmos. A partir de este tema aprenderemos a definir nuestras propias clases, crear objetos a partir de ellas y determinar qué información y operaciones corresponden a cada uno.

Conviene señalar que la evolución histórica de la POO no consiste en la sustitución absoluta de un paradigma por otro. Los distintos enfoques de programación pueden complementarse, y los lenguajes actuales combinan características procedentes de diferentes tradiciones. La orientación a objetos no elimina la necesidad de utilizar secuencias, selecciones, repeticiones o métodos: aporta una forma adicional de organizar los datos y las operaciones que intervienen en la solución de un problema.

La filosofía de la POO

Más allá de su evolución histórica, La POO parte de una premisa fundamental: **el programa refleja las entidades que participan en el problema y sus reponsabilidades, como en el mundo real**.

No procesamos la realidad como un conjunto de variables numéricas inconexas ni como un flujo ininterrumpido de instrucciones. Los seres humanos percibimos la realidad identificando **objetos** —un coche, una cuenta bancaria, un estudiante, una factura—, que poseen unas características determinadas y son capaces de realizar o responder a una serie de acciones concretas.

Por ejemplo, en una aplicación de gestión bancaria podemos representar una cuenta mediante la información que permite identificarla y conocer su saldo. También podemos asociar a esa cuenta las operaciones necesarias para ingresar dinero, retirar una cantidad o consultar el saldo disponible. Cada cuenta concreta tendrá su propio estado, aunque todas las cuentas puedan compartir una estructura y unas operaciones comunes.

La POO se apoya así en varias ideas que desarrollaremos a lo largo del tema. La primera es que un objeto representa una entidad que posee características y puede realizar determinadas acciones. La segunda es que los objetos interactúan con otros objetos para colaborar en la resolución de un problema. La tercera es que debemos definir con claridad qué responsabilidades corresponden a cada entidad y cómo puede utilizarse desde el resto del programa.

Un objeto no tiene por qué representar necesariamente una entidad física del mundo real, como un vehículo o una cuenta bancaria. También puede representar conceptos abstractos, como una operación matemática, una configuración o una sesión de trabajo. Lo importante es que la representación resulte útil para el problema que deseamos resolver.

Tampoco debemos pensar que orientar un programa a objetos consiste simplemente en crear muchas clases. La elección de las entidades que necesitamos representar, qué información concreta necesitamos conservar de cada una, la distribución de las responsabilidades y las relaciones que establecemos entre ellas son decisiones de diseño que influyen directamente en la claridad y la mantenibilidad del software.

A continuación, analizaremos qué aportan los objetos para abordar problemas que pueden surgir si solo usamos programación estructurada. Después estudiaremos las características que permiten describirlos y los principios que regulan la forma en que representamos y protegemos su información.

Límites de la programación estructurada

La **programación estructurada**, consolidada en los setentas y ochentas gracias a investigadores como Dijkstra, Wirth y Hoare, supuso un avance respecto a la programación no estructurada, proporcionándonos herramientas para organizar el flujo de control de un programa mediante secuencias, selecciones y repeticiones. Además, la utilización de procedimientos y funciones permite dividir un problema complejo en tareas más pequeñas (diseño descendente o *top-down*), de manera que cada una de ellas pueda diseñarse, implementarse y comprobarse de forma independiente. La programación estructurada permitió elevar sustancialmente la calidad y legibilidad del código.

Sin embargo, aunque en un programa sencillo la separación entre los datos y los procedimientos que los manipulan no suele plantear grandes problemas, a medida que los sistemas crecen en dimensiones y complejidad, aparece una dificultad que no siempre resulta sencillo resolver únicamente mediante la descomposición del algoritmo en subprogramas: **cómo organizar la información y las operaciones que actúan sobre ella**. La causa fundamental no estriba en una deficiencia sintáctica, sino en una desconexión metodológica de fondo: **la programación estructurada mantiene separados los datos de los algoritmos que los manipulan**.

En el modelo estructurado, las estructuras de datos se definen de forma pasiva, mientras que las funciones y procedimientos actúan de manera activa desde el exterior: **la información que describe una entidad y las operaciones que la manipulan se organizan como elementos independientes**. Los distintos procedimientos necesitan acceder a los datos que les corresponden y, en muchos casos, deben conocer cómo están representados para poder utilizarlos correctamente. Esta dependencia puede extenderse a distintas partes del programa, de modo que una modificación en la representación de los datos obliga a revisar todos los procedimientos afectados.

Imaginemos que deseamos desarrollar una aplicación bancaria que deba gestionar numerosas cuentas. Cada una de ellas posee información propia, como su número identificativo y su saldo, y está sujeta a determinadas operaciones y condiciones. Si, posteriormente, decidimos incorporar a cada cuenta una nueva característica que deba tenerse en cuenta al retirar dinero, tendremos que identificar los procedimientos que intervienen en esa operación y comprobar que todos aplican las nuevas condiciones. Si la lógica relacionada con una misma entidad se encuentra dispersa entre numerosos subprogramas, resulta más difícil localizar todas las dependencias y verificar que los cambios no introducen inconsistencias.

Esta dificultad está relacionada con el **acoplamiento**: grado de dependencia que existe entre las diferentes partes de un programa. Cuanto mayor sea la dependencia entre sus componentes, más probable será que una modificación aparentemente localizada obligue a revisar otras partes del sistema. El problema no consiste simplemente en que el programa tenga muchos procedimientos, sino en que sus responsabilidades y sus dependencias están distribuidas de manera que resulta difícil comprender cómo se relacionan entre sí.

También debemos considerar la posibilidad de que diferentes procedimientos modifiquen los mismos datos. Si cada uno de ellos puede alterar el estado de una cuenta, tendremos que asegurarnos de que todos respeten las condiciones necesarias para mantenerla en un estado válido. Una operación que inicialmente estaba diseñada para retirar dinero podría dejar de cumplir sus requisitos si otro procedimiento modifica el saldo sin aplicar las mismas comprobaciones. La solución exige establecer y mantener reglas que no siempre quedan ligadas a un único componente del programa.

De manera similar, podemos reutilizar un procedimiento siempre que su funcionamiento se adapte a los datos que recibe y a las condiciones en las que debe trabajar. Sin embargo, cuando los datos y las operaciones relacionadas con una entidad se encuentran dispersos, reutilizar todos los elementos necesarios puede exigir conocer numerosos detalles de su organización interna. Además, una modificación destinada a una parte de la aplicación puede afectar a otra que utiliza los mismos procedimientos con un propósito diferente.

Recapitulando, los principales problemas de la programación estructurada son:

1. Representación: dificultad para gestionar de manera coordinada múltiples variables.
2. Acoplamiento: cualquier modificación exige modificar todo el código que se ve afectado.
3. Reutilización: la dispersión del código limita la reutilización efectiva.

Acoplamiento: Nivel de dependencia directa existente entre los procedimientos de un programa y sus variables.

Efecto dominó: Fenómeno por el cual una modificación en la estructura interna de los datos obliga a rehacer o ajustar múltiples funciones distribuidas a lo largo de todo el código fuente. Es denominado técnicamente acoplamiento fuerte (*tight coupling*) o propagación de errores.

Estas dificultades **no son inevitables** en la programación estructurada. Una descomposición adecuada, unas interfaces bien definidas y el control de acceso a los datos permiten reducir las dependencias y desarrollar aplicaciones de considerable complejidad. El problema es que la organización del algoritmo en subprogramas no garantiza, por sí sola, que la información de cada entidad y las operaciones que le corresponden queden agrupadas y protegidas de manera coherente.

La aportación fundamental de la POO consiste en **proporcionar un nivel de organización superior**. En lugar de mantener necesariamente separados los datos y los procedimientos que los manipulan, permite reunir la información y las operaciones relacionadas con una misma entidad en una unidad de organización: el **objeto**. De esta forma, cada objeto puede representar una entidad concreta del problema y asumir las responsabilidades que le corresponden, mientras que el resto del programa interactúa con él mediante las operaciones que ofrece; y si la representación interna de un dato cambia, solo los procedimientos del propio objeto necesitan ajustarse.

Esta organización no elimina la necesidad de utilizar variables, expresiones, selecciones, repeticiones o funciones, ni garantiza que un programa esté bien diseñado. Aporta un criterio adicional para distribuir las responsabilidades y controlar las dependencias: organizar parte de la solución en torno a las entidades que intervienen en el problema, en lugar de atender únicamente a las operaciones que deben ejecutarse.

A continuación, estudiaremos qué caracteriza a un objeto. En particular, analizaremos qué información mantiene, qué operaciones puede realizar y cómo podemos distinguir una entidad concreta de las demás. Estos conceptos nos permitirán comprender las nociones de **estado, comportamiento e identidad**, que constituyen el siguiente paso en el estudio del paradigma orientado a objetos.

Modelo de objeto

Cuando aplicamos el paradigma orientado a objetos, no basta con identificar las entidades que intervienen en el problema. También debemos determinar qué información las describe, qué operaciones pueden realizar y cómo podemos distinguir unas de otras. Un **objeto** es una entidad del mundo real; un ente discreto y autónomo del dominio del problema que se caracteriza por tres características fundamentales: **estado**, **comportamiento** e **identidad**.

Estos tres conceptos, formulados originalmente por Grady Booch en sus trabajos pioneros sobre análisis y diseño orientado a objetos, constituyen la piedra angular sobre la que se fundamenta la abstracción de objetos. Aunque están estrechamente

relacionadas, cada una representa un aspecto diferente de la entidad que deseamos modelar.

Estado

El **estado** (*state*) de un objeto representa su condición en un momento concreto de la ejecución del programa. Está constituido por el conjunto de valores que describen sus características, sus **atributos**, en un momento determinado. Estos valores representan la información que necesitamos conservar sobre la entidad y permiten conocer su situación actual.

Una cuenta bancaria, por ejemplo, posee información que permite identificarla, a su titular y determinar el saldo. Dicho saldo es especialmente significativo porque puede cambiar durante la ejecución de la aplicación: un ingreso incrementa su valor, mientras que una retirada lo reduce. Por tanto, el estado de la cuenta antes de realizar una operación puede ser diferente del que presenta después de ejecutarla.

Esto significa que **el estado no es necesariamente inmutable**. Las características que describen un objeto pueden permanecer constantes o modificarse por operaciones que se realizan sobre el mismo. Es decir, el estado representa los valores que caracterizan al objeto en cada momento, no solamente los que tenía cuando comenzó a utilizarse.

Además, el estado debe interpretarse en relación con la entidad concreta que representa. Si tenemos varias cuentas, cada una mantendrá su propio saldo. Aunque dos cuentas presenten inicialmente la misma cantidad de dinero, sus estados pueden evolucionar de manera diferente si sobre ellas se realizan operaciones distintas.

Un aspecto crucial en el diseño de objetos es la noción de **estado válido**. No se debe permitir jamás que el estado de un objeto adopte valores incoherentes o imposibles dentro del dominio del problema (como un saldo alfanumérico). Como veremos, la encapsulación va a jugar aquí un papel fundamental.

Comportamiento

El **comportamiento** (*behavior*) de un objeto determina cómo actúa y reacciona ante las peticiones externas. Comprende las operaciones que puede realizar o las respuestas que puede ofrecer cuando otros componentes del programa interactúan con él. Mientras que el estado describe cómo se encuentra una entidad, el comportamiento determina qué podemos hacer con ella.

Una cuenta bancaria puede permitir ingresar dinero, retirarlo o consultar el saldo disponible. Estas operaciones representan comportamientos relacionados con la cuenta, puesto que expresan las tareas que podemos realizar utilizando la información que mantiene. Las operaciones no tienen que modificar necesariamente el estado del objeto: un ingreso o retirada produce cambios en el saldo pero una consulta puede limitarse a proporcionar información sin alterar los valores almacenados. Por tanto, el

comportamiento comprende tanto las operaciones que modifican el estado como aquellas que permiten conocerlo o utilizarlo.

También debemos considerar que las operaciones deben respetar unas condiciones. Por ejemplo, si no se permite retirar una cantidad superior al saldo, el comportamiento asociado a la retirada tendrá que comprobar esa condición antes de modificar el estado. De esta manera, las operaciones no se limitan a ejecutar cálculos: también expresan reglas que determinan cómo puede utilizarse la entidad.

En la programación orientada a objetos, **el estado y el comportamiento están relacionados**. Las operaciones de un objeto pueden consultar o modificar su estado, y el estado actual puede condicionar el resultado de esas operaciones. Esta relación permite reunir la información y las responsabilidades que corresponden a una misma entidad, en lugar de tratarlas necesariamente como elementos independientes.

En la filosofía de la orientación a objetos, realizar una acción asociada al comportamiento se concibe bajo el modelo de **paso de mensajes**. Un objeto emisor (*cliente*) envía un mensaje a un objeto receptor (*servidor*) solicitando la ejecución de una tarea. El objeto receptor procesa la petición de acuerdo con sus reglas, modifica su estado si procede y, opcionalmente, devuelve un resultado al emisor.

Identidad

La **identidad** (*identity*) es la propiedad de un objeto que lo distingue de manera inequívoca de todos los demás objetos del programa. Permite distinguir un objeto concreto de los demás; representa el hecho de que cada objeto corresponde a una entidad individual, incluso cuando la totalidad de sus atributos contengan valores idénticos.

Por ejemplo, dos cuentas pueden tener el mismo saldo y permitir realizar las mismas operaciones. Esta coincidencia no significa que ambas cuentas sean la misma: cada una conserva su propia identidad y las operaciones realizadas sobre una deben afectar únicamente a esa cuenta.

Es fundamental distinguir con total claridad entre **identidad** e **igualdad de estado** (o equivalencia). El estado está formado por los valores que describen al objeto y puede cambiar a lo largo del tiempo; la identidad, en cambio, permite reconocer a un objeto concreto aunque sus valores se modifiquen. De este modo, una cuenta sigue siendo la misma cuenta cuando su saldo pasa de cien a doscientos euros, aunque su estado haya cambiado.

Tampoco debemos confundir la identidad con una característica particular utilizada para identificar una entidad dentro de una aplicación. El número de cuenta, por ejemplo, es un dato que permite reconocerla en el sistema bancario, pero la idea de identidad es más general: se refiere a la individualidad del objeto, no exclusivamente al valor de uno de sus datos.

Esta distinción adquiere especial relevancia cuando los programas gestionan muchas entidades semejantes. Podemos tener miles de cuentas semejantes pero necesitamos

tratarlas como objetos individuales para dirigir cada operación a la entidad que corresponda.

Objeto: Entidad que encapsula estado y comportamiento, teniendo una existencia única y diferenciada.

Estado: Conjunto de valores que describen las características de un objeto en un momento determinado.

Comportamiento: Conjunto de acciones que un objeto puede realizar en respuesta a mensajes recibidos de otros objetos.

Identidad: Característica que permite distinguir un objeto concreto de los demás, independientemente de si comparten estado y comportamiento.

Relación entre estado, comportamiento e identidad

Si queremos retirar dinero de una cuenta, debemos primero determinar de qué cuenta, pues su identidad la distingue del resto. La operación tendrá en cuenta su estado actual, especialmente el saldo, para comprobar si podemos hacerlo. Si la operación se completa, cambiará su estado, pero la identidad de la cuenta no.

Esta forma de representar las entidades permite razonar sobre cómo funciona un programa a partir de objetos que mantienen información propia y ofrecen operaciones relacionadas con ellos mismos. En lugar de considerar una colección de variables y procedimientos, podemos identificar qué representa a cada entidad, qué responsabilidades tiene cada una y cómo deben colaborar para resolverlo.

Propiedad	Pregunta a la que responde	Rol en el sistema
Estado	¿Qué sabe o contiene el objeto?	Información interna en un momento dado.
Comportamiento	¿Qué puede hacer el objeto?	Algoritmos y servicios que transforman (o no) el estado.
Identidad	¿Quién es el objeto?	Distingue instancias concretas e independientes.

Conviene recordar que estas características constituyen un modelo conceptual, no una garantía de que el diseño sea correcto. Para construir el programa debemos decidir qué información resulta relevante, qué operaciones necesita cada objeto y cómo se relaciona con el resto del sistema. Una elección inadecuada puede generar objetos con responsabilidades poco claras o dependencias innecesarias.

A continuación, estudiaremos dos principios que ayudan a tomar estas decisiones: **la abstracción** y **la encapsulación**.

Estos dos conceptos representan los dos primeros **pilares fundamentales** de la programación orientada a objetos. No basta con identificar el estado, el comportamiento y la identidad: hay que decidir qué características necesitamos conservar, qué comportamientos nos interesan y cómo organizamos estos elementos para usarlos coherentemente.

Aunque abstracción y encapsulación están estrechamente relacionados y suelen mencionarse de forma conjunta en la literatura, responden a preguntas diferentes. La abstracción nos ayuda a determinar qué aspectos de una entidad son relevantes para el problema que deseamos resolver. La encapsulación permite reunir la información y sus operaciones relacionadas, separando su funcionamiento interno de la forma en que la usa el resto del programa.

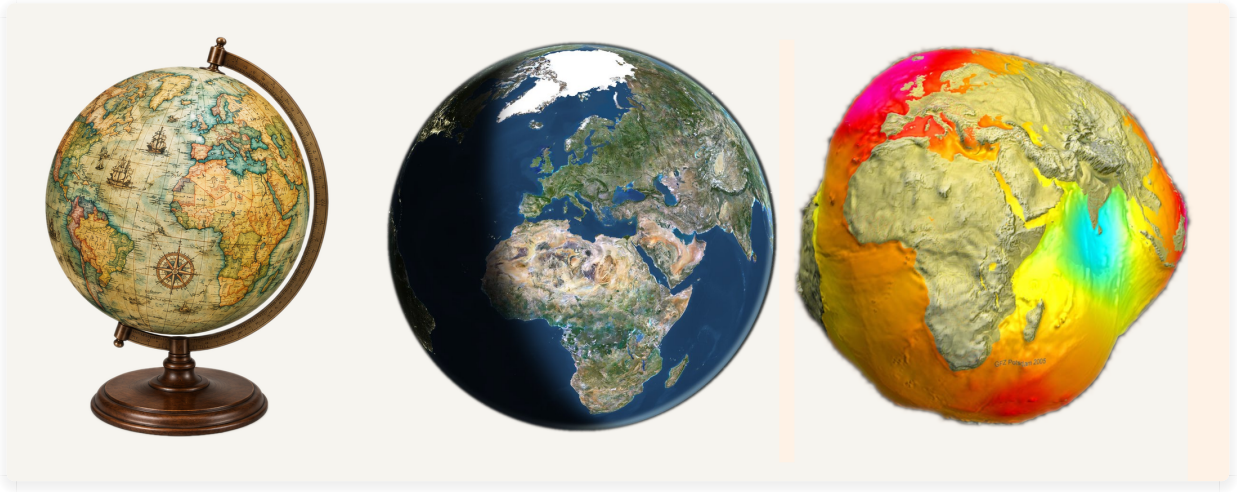
La aplicación de estas dos herramientas permite gestionar la complejidad de los grandes sistemas informáticos, liberando al desarrollador de la necesidad de conocer minuciosamente los detalles de cada módulo para poder utilizarlo con total garantía.

Abstracción

La mente humana es incapaz de procesar simultáneamente la totalidad de la complejidad del mundo real. Para interactuar con nuestro entorno, las personas abstraemos de forma inconsciente la realidad, creando modelos simplificados que contienen únicamente la información necesaria para cumplir un propósito concreto.

La **abstracción** es el proceso mediante el cual identificamos las características y los comportamientos relevantes de una entidad, dejando aquellos detalles que no resultan necesarios para el problema que estamos resolviendo. No consiste en describir exhaustivamente la realidad, sino en construir una representación simplificada que permita trabajar con ella de manera eficaz.

Para comprenderlo, podemos pensar en la representación de la Tierra. Estamos ya acostumbrados a ver imágenes tomadas desde el espacio en las que vemos una esfera, debido a que las diferencia de distribución de masa y gravedad son mínimas, aunque sabemos que no es exactamente así, sabemos que es un esferoide achatado por los polos. Sin embargo, cuando la representamos, solemos usar lo que denominamos un globo terráqueo, una esfera perfecta que pude representar líneas para las fronteras, ríos, un relieve exagerado (si lo tiene), pero que nos ayuda a comprender cómo es, dónde estás los accidente geográficos o los países. Y si estudiamos cómo varía la fuerza de la gravedad en diferentes puntos del planeta, usaremos la *patata de Postdam*, una imagen irregular y abultada que exagera la escala vertical unas 10000 veces.



Este ejemplo muestra que **la abstracción depende del problema que deseamos resolver**. Una misma entidad del mundo real puede representarse de diferentes maneras, cada una adecuada a un contexto concreto. No existe necesariamente una única representación válida de un vehículo o una persona: debemos elegir la información y las operaciones que realmente necesita la aplicación.

En el caso de la cuenta bancaria que hemos utilizado anteriormente, podríamos representarla mediante su número identificativo, el titular y el saldo disponible. Las operaciones relevantes serían ingresar dinero, retirar una cantidad y consultar el saldo. Para gestionar las operaciones habituales de una cuenta no necesitaríamos representar todos los detalles de la infraestructura informática del banco ni los mecanismos internos que permiten almacenar sus datos.

La abstracción también permite trabajar con una entidad sin conocer todos los detalles de su funcionamiento. Cuando utilizamos la cuenta, nos interesa saber qué operaciones podemos realizar y cuáles son sus resultados; no necesitamos conocer ni la organización ni todos los pasos internos que se ejecutan para completar una operación.

Por tanto, podemos afirmar que **abstraer consiste en identificar qué necesitamos conocer de una entidad y qué necesitamos hacer con ella**, atendiendo a los objetivos de la aplicación. Esto permite reducir la complejidad del problema, concentrar la atención en los aspectos importantes y evitar que la representación incluya información innecesaria.

Encapsulación

Una vez que hemos determinado la información y operaciones que debe contemplar nuestra representación, debemos decidir cómo organizarlas: la **encapsulación** es el mecanismo técnico de implementación que materializa dicha abstracción en el código. Consiste en agrupar los datos y las operaciones relacionados con una misma entidad y delimitar la forma en que el resto del programa puede acceder a su información e interactuar con la misma. Etimológicamente, significa «cerrar dentro de una cápsula»; esa cápsula es la **clase**.

Recordemos uno de los problemas que analizamos al estudiar los límites de la programación estructurada: cuando los datos y los procedimientos que los manipulan se encuentran dispersos por el programa, diferentes componentes pueden depender de la representación interna de esos datos. La encapsulación permite abordarlo reuniendo en un mismo objeto la información que describe a la entidad y las operaciones correspondientes. De este modo, los datos y las operaciones dejan de ser elementos independientes y pasan a formar parte de una unidad con una responsabilidad determinada.

Sin embargo, encapsular no significa únicamente agrupar datos y operaciones; implica también establecer límites al acceso a la información y determinar cómo puede utilizarse desde otras partes del programa. Garantizamos que un objeto sea robusto evitando que cualquier otro componente pueda modificar su estado sin respetar las condiciones que regulan su funcionamiento. A esto lo denominamos **ocultación de información** (*Information Hiding*).

Supongamos que el saldo de la cuenta bancaria pudiera modificarse directamente desde cualquier parte de la aplicación, distintos procedimientos podrían incrementar o reducir su valor sin comprobar las condiciones necesarias. Un código aplicaría correctamente la restricción mientras otro podría permitir una retirada superior al saldo disponible, por ejemplo. Para que el sistema fuera coherente, todos los procedimientos deberían estar implementados de forma correcta y mantenerse actualizados. Con la encapsulación, podemos gestionar que las operaciones destinadas a modificar el saldo comprueben las condiciones correspondientes y el resto del programa deba solicitar que realizar una operación, en lugar de modificar directamente la información interna sin atender a las reglas.

Otra ventaja de la encapsulación, más concretamente de la ocultación, reside en la actualización de la información interna. Supongamos que queremos representar un punto del plano euclideo e, inicialmente, lo hacemos mediante sus coordenadas cartesianas pero, posteriormente, decidimos modificarlo a las coordenadas polares. Si el resto del programa depende directamente de esa representación, tendremos que revisar todo el código que la utiliza. Pero, si interactúa mediante operaciones claramente definidas, podremos realizar el cambio interno sin necesidad de modificarlo todo, siempre que se mantenga su funcionamiento observable.

La encapsulación contribuye así a reducir el acoplamiento y a mantener la coherencia con la entidad. No elimina por completo las dependencias ni garantiza que el diseño sea correcto, pero proporciona mecanismos para controlarlas y facilita que cada objeto asuma las responsabilidades que le corresponden.

Sinergia entre abstracción y encapsulación

Aunque ambas tienen finalidades distintas, se complementan en el diseño orientado a objetos: la abstracción se centra en el comportamiento observable de un objeto, mientras que la encapsulación se encarga de la estructura interna que hace posible ese comportamiento.

Criterio	Abstracción	Encapsulación
Propósito	Simplificar la realidad seleccionando lo esencial.	Proteger el estado y ocultar los detalles de implementación.
Fase principal	Análisis y Diseño de clases.	Implementación y Codificación.
Enfoque	Centrada en el **Qué hace** el objeto.	Centrada en el **Cómo lo hace** y en **Cómo se almacena**.
Resultado	Definición de la interfaz pública de la clase.	Aislamiento del estado mediante modificadores como `private`.

Gracias a ambos principios, el resto de la aplicación puede trabajar con la entidad sin necesitar conocer todos sus detalles internos. El código que solicita una operación debe identificar la entidad concreta y pedir que se efectúe dicha operación; no necesita reproducir en cada lugar todos los mecanismos internos ni conocer cómo es la abstracción.

Abstracción: Proceso de identificar y seleccionar las características y los comportamientos relevantes de una entidad para construir una representación adecuada al problema que se desea resolver.

Encapsulación: Principio que permite reunir los datos y las operaciones relacionados con una entidad y delimitar cómo puede accederse a su información e interactuar con ella desde el resto del programa.

Detalle de implementación: Aspecto interno de la representación o del funcionamiento de un componente que no necesita conocer el resto del programa para utilizarlo correctamente.

Ocultación (*Information Hiding*): Principio derivado de la encapsulación que dictamina que los detalles de implementación interna de un módulo deben permanecer ocultos tras una interfaz pública estable.